

Signal Desk Capstone Evidence Report

Functionality deployment architecture and rubric evidence

Author	Srinivas Malyala
Application	Signal Desk AI Stock Market Research Assistant
Verified user	malyalasrinivas@gmail.com
Live application	signal-desk-frontend-s88i.onrender.com
Evidence date	October 3, 2026

Submission purpose. This report gives graders direct evidence for every rubric category and shows how the working application, data platform, agent actions, analytics, deployment, and Big Data characteristics fit together.

Primary conclusion. The available code, accepted workspace results, live deployment, and current automated verification support the top rubric band in all eight categories. The report distinguishes live observations from repository acceptance evidence.

Live link. [Open Signal Desk](#)

Project Overview

Signal Desk is an authenticated stock research application that combines live market data, SEC and article evidence, an action-taking MCP agent, user-owned Lakebase state, and Databricks analytics. A researcher can inspect price performance, compare peers, retrieve attributable filing and news passages, maintain a watchlist, save research, and review operational analytics from one interface.

Evidence area	Demonstrated result	Primary proof
Scale	1,255,489 unique Silver market rows	Accepted market pipeline reconciliation
Variety	Structured, semi-structured, and unstructured data	OHLCV, XBRL, JSON, filings, and news
Agent	Six retrieval and three confirmed write tools	MCP contracts and 10 of 10 Supervisor evaluation
Analytics	Five Gold metric families	Lakebase history through Bronze and Silver to Gold
Quality	285 tests and Ruff pass	Local verification on October 3, 2026
Deployment	Frontend and MCP live on Render	Live application and Render deployment evidence

Architecture

Render hosts the Flask frontend and FastMCP application tier. Databricks owns the Spark pipelines, Delta tables, SQL analytics, AI Search, and Agent Bricks Supervisor. Lakebase provides operational storage and serving copies. Short-lived signed assertions carry the authenticated browser identity to MCP, while separate machine credentials protect platform-to-platform calls.

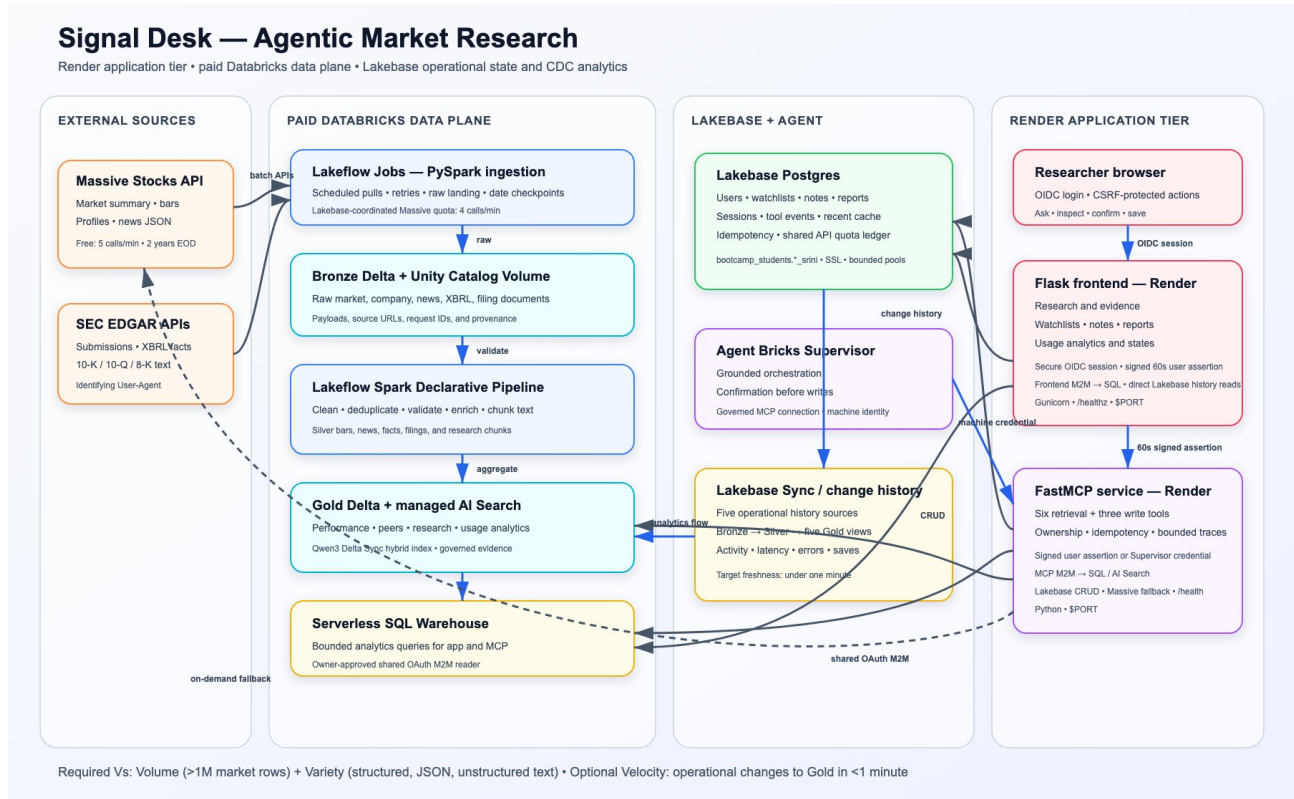


Figure 1 Signal Desk split-host architecture

Rubric Evidence Summary

The table below identifies the highest rubric band supported by the submitted evidence. It is an evidence map, not a guaranteed grade. Detailed implementation and verification references follow.

Category	Points	Top-band evidence	Where to verify
Spark data pipeline	15	Reusable Spark ingestion, validation, enrichment, quarantine, and Gold outputs	pipelines and resources
Third-party API	10	Massive and SEC integrations with secrets, retries, caching, validation, and shared quota controls	clients, ingestion, migration 0005
Lakebase model	15	Normalized operational model with keys, constraints, indexes, ownership, audit fields, and idempotency	migrations and lakebase modules
Action-taking agent	20	Grounded retrieval plus three protected writes; deployed Supervisor passes 10 of 10 cases	MCP server, prompt, evaluations
Analytics pipeline	10	Incremental history processing into privacy-safe Silver and five useful Gold metric families	analytics pipeline and acceptance tool
Frontend workflow	10	Authenticated end-to-end research, watchlist, memory, evidence links, errors, and analytics	live screenshots and dashboard code
Deployed application	5	Live Render frontend and MCP with OIDC, health checks, Blueprint configuration, and runbook	live URL, render.yaml, runbook
Two Big Data Vs	15	Volume above one million rows and meaningful unstructured document processing	coverage results and research index

Spark Data Pipeline 15 points

Separate Lakeflow Spark Declarative Pipelines process market, SEC, article, document, and activity data. The transformations use explicit schemas, deterministic keys, deduplication, quarantine paths, reconciliation, and reusable Bronze, Silver, and Gold layers.

Implementation Evidence

- Market processing: pipelines/market_bars_classified.py, silver_market_bars.py, silver_market_quarantine.py, and gold_market_data_coverage.py.
- Research processing: bronze_sec_company_facts.py, bronze_sec_filing_documents.py, silver_sec_facts.py, silver_research_articles.py, and silver_research_chunks.py.
- Deployment definitions: resources/market_pipeline.pipeline.yml and resources/research_pipeline.pipeline.yml.

Demonstrated Result

- 1,255,677 Bronze market rows reconciled to 1,255,489 unique Silver rows plus 188 deterministic quarantines across 81 manifest dates.
- The bounded research run produced 2 companies, 12 filings, 57,806 facts, 86 articles, 549 article-ticker links, and 507 traceable chunks with zero integrity violations.
- Cached research ingestion reran without external API calls, demonstrating a repeatable workflow.

Third Party API Integration 10 points

Massive Stocks supplies company, price, fundamentals, and news data. SEC EDGAR supplies company submissions, filing documents, and structured facts. Keys and identifying contacts remain in secrets or runtime configuration rather than source control.

Implementation Evidence

- `mcp_server/massive_client.py` implements bounded calls, response validation, retries, plan-aware errors, and safe result envelopes.
- `ingestion/market_backfill.py`, `ingestion/sec_client.py`, and `ingestion/article_landing.py` provide immutable landing, checkpoints, and cached reruns.
- `migrations/0005_massive_rate_limit.sql` and the shared Lakebase ledger coordinate rate limits across Render and Databricks jobs.

Demonstrated Result

- A cross-host acceptance run recorded four Render attempts and one job attempt, never exceeded four acquisitions in a rolling minute, and delayed attempt five by 60.225 seconds.
- Malformed or unavailable source data is labeled safely; plan-restricted values are never converted to zero or fabricated.

Lakebase Data Model 15 points

The shared PostgreSQL schema uses suffix-isolated operational tables for users, watchlists, companies, price snapshots, articles, notes, reports, agent events, traces, quota attempts, and serving copies. Versioned migrations are idempotent and checksum protected.

Implementation Evidence

- `mcp_server/migrations/0001_operational_core.sql` through `0005_massive_rate_limit.sql` define the model, CDC settings, indexes, and operational fields.
- `mcp_server/lakebase.py` and `mcp_server/lakebase_serving.py` provide bounded pools, ownership-scoped queries, and serving access.
- `mcp_server/action_service.py` enforces validation, confirmation, idempotency, and changed-payload rejection.

Demonstrated Result

- PostgreSQL 17 or newer connectivity, 16 owned tables, five migrations, and six full-replica-identity tables were verified.
- Two real Google principals proved isolated watchlists, notes, reports, and traces through the deployed UI; disposable state was then removed exactly.

Action Taking AI Agent 20 points

FastMCP exposes six retrieval tools and three protected write tools. The Agent Bricks Supervisor selects tools, validates requests, links evidence, asks for confirmation before consequential actions, and reports bounded failures without inventing results.

Implementation Evidence

- Retrieval tools cover price performance, company research, peer comparison, watchlists, semantic evidence, and notable updates.
- Write tools add or remove watchlist items and save research notes or analysis reports in Lakebase.
- agent/system_prompt.md defines source selection, ambiguity handling, confirmation, idempotency, and investment-advice boundaries.

Demonstrated Result

- Exact replays return the original action result; the same idempotency key with a changed payload is rejected.
- The deployed governed Supervisor passed all 10 live evaluation cases and exposes the exact nine-tool MCP contract.
- Every research card includes a source type, date, title, and link for user inspection.

Analytics Pipeline 10 points

Lakebase history tables feed an incremental Databricks analytics pipeline. Bronze preserves bounded change images, Silver deduplicates and pseudonymizes activity, and Gold tables calculate user, tool, error, latency, watchlist, and research-save metrics.

Implementation Evidence

- pipelines/bronze_lakebase_changes.py and pipelines/silver_agent_activity.py implement the incremental foundation.
- Gold modules cover daily active researchers, tool latency, error rate, watchlist changes, and research saves.
- tools/phase6_cdf_acceptance.py verifies exact change counts, latency, aggregation, and cleanup.

Demonstrated Result

- A self-cleaning transaction produced 18 source history rows, 34 Bronze rows, 26 effective Silver rows, and exact results across five Gold metric families.
- Maximum reported source latency was 85 seconds. The submission claims Volume and Variety for the Big Data score rather than claiming sub-minute Velocity.

Frontend and Core Workflow 10 points

The responsive Flask interface supports authenticated research, market and peer workflows, source inspection, watchlist actions, research memory, current signals, and usage analytics. Consequential writes have explicit controls and bounded feedback.

Implementation Evidence

- dashboard/templates/index.html and dashboard/static/app.js implement the complete interaction model.
- dashboard/auth.py enforces Google OIDC identity; dashboard/mcp_client.py creates a fresh signed assertion for each MCP session.
- Frontend and route contracts are covered by tests/test_dashboard_routes.py, tests/test_dashboard_ui_contract.py, and tests/test_render_frontend_routes.py.

Demonstrated Result

- The live application showed the expected signed-in user and loaded attributable research, an owned watchlist, saved notes and reports, latest signals, and Gold analytics.
- The interface surfaced an explicit stale-data status while preserving the most recent processed analytics, demonstrating a useful degraded state.

Deployed Application 5 points

The frontend and FastMCP server run as separate Render services. Databricks retains the data and processing plane. The deployment contract includes health checks, reproducible builds, secret placeholders, OIDC configuration, and an operations runbook.

Implementation Evidence

- `render.yaml` defines both Python services and contains no Databricks data resources.
- `docs/RENDER_DEPLOYMENT_RUNBOOK.md` and `docs/RENDER_OIDC_PREPARATION.md` document setup, configuration, secret entry, validation, and recovery.
- `docs/SPLIT_WORKSPACE_APP_DEPLOYMENT_PLAN.md` records the boundary between Render and Databricks.

Demonstrated Result

- The frontend was accessible at the submitted URL under the verified Google account.
- Render showed `signal-desk-mcp` as Live, Blueprint managed, and linked to the deployed Git commit.
- Both services use Free instances; they must be pre-warmed because cold starts may add 50 seconds or more.

Big Data Volume and Variety 15 points

The project demonstrates two rubric-defined Big Data characteristics with measurable downstream use. Volume comes from the distributed market dataset. Variety comes from processing structured market and XBRL data, semi-structured source payloads, and unstructured filing and news text.

Implementation Evidence

- The Spark market pipeline processes more than one million rows and publishes certified market history used by research workflows.
- Document text is normalized, section-aware, chunked, embedded, indexed, filtered, reranked, and returned with provenance.
- The accepted AI Search evaluation used 51 cases against a 393-row canonical serving corpus.

Demonstrated Result

- Volume: 1,255,489 unique Silver market rows after deterministic duplicate and malformed-record handling.
- Variety: 507 traceable filing and article chunks in the bounded research pipeline and 393 canonical search documents in the accepted index.
- Search quality: Recall at 5 of 1.0000, MRR of 0.9902, nDCG at 5 of 0.9928, zero provenance failures, and zero filter violations.

Live Application Evidence

The screenshots in this section were captured from the deployed application and Render dashboard on October 3 and 4, 2026. They contain no credentials, access tokens, connection strings, or private research bodies.

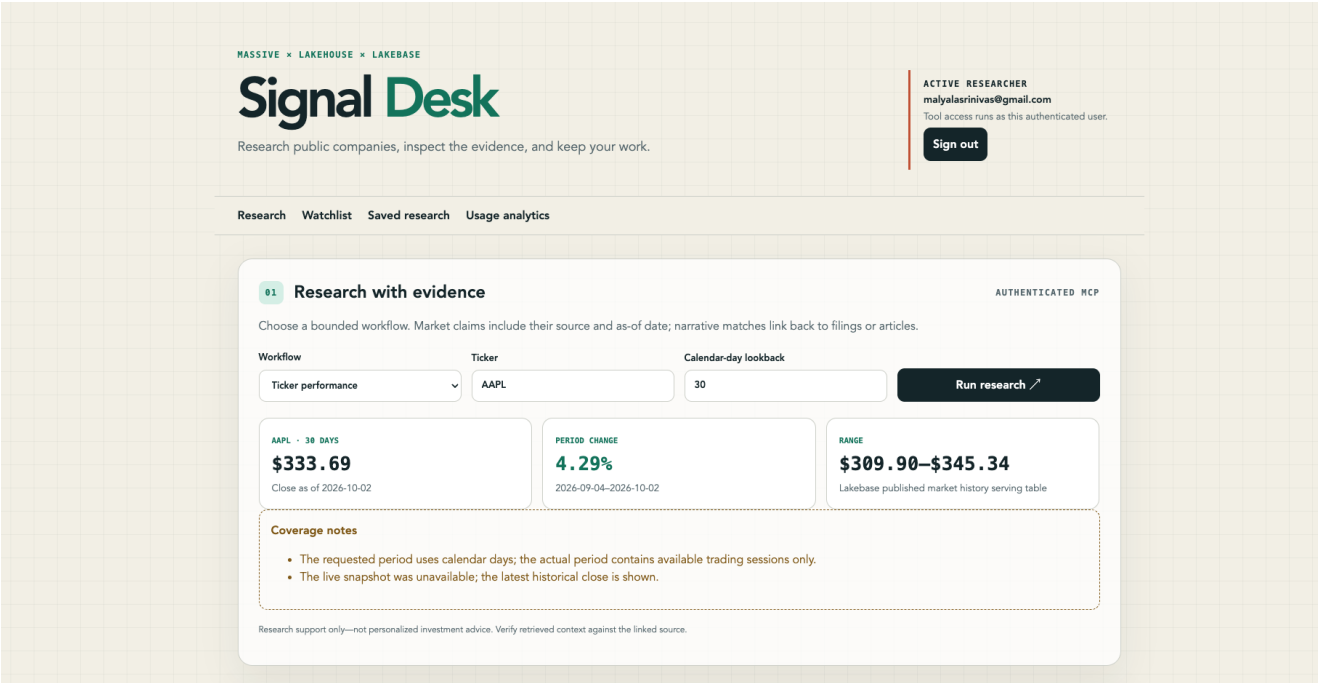


Figure 2 Authenticated ticker performance workflow

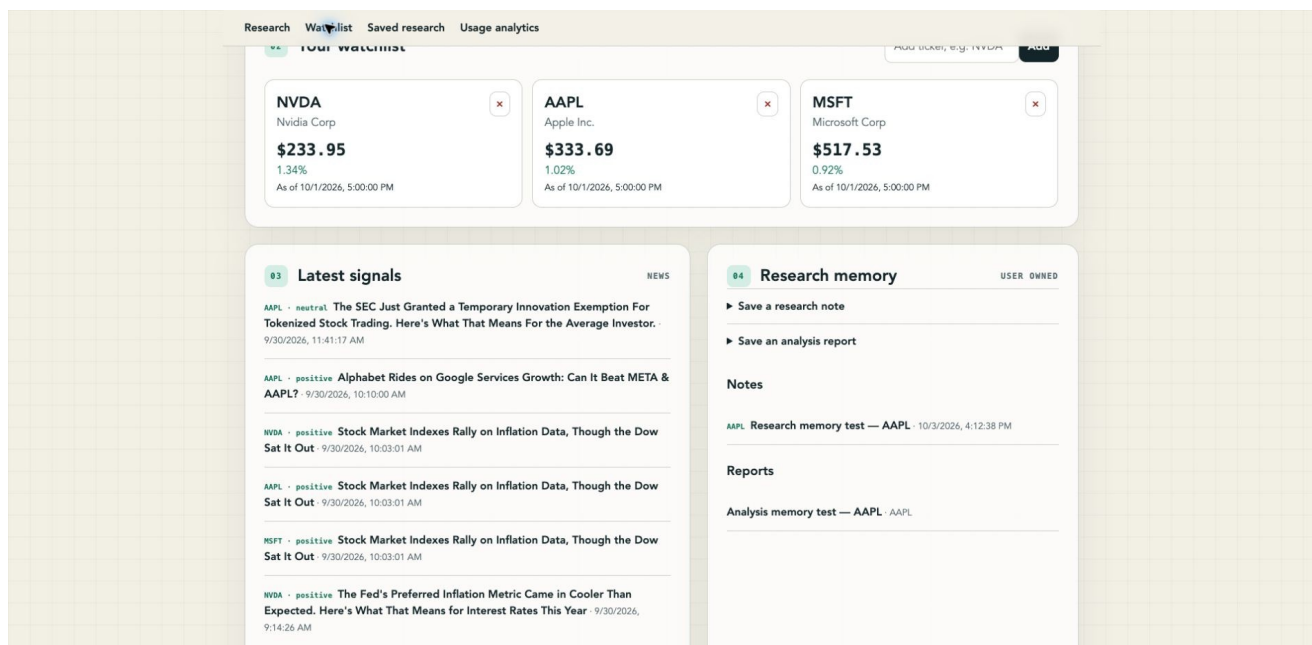


Figure 3 User-owned watchlist signals notes and reports

The watchlist combines the latest locally served price with an as-of timestamp. The adjacent research memory confirms that notes and reports are retained for the authenticated user. Signals link to the underlying articles.

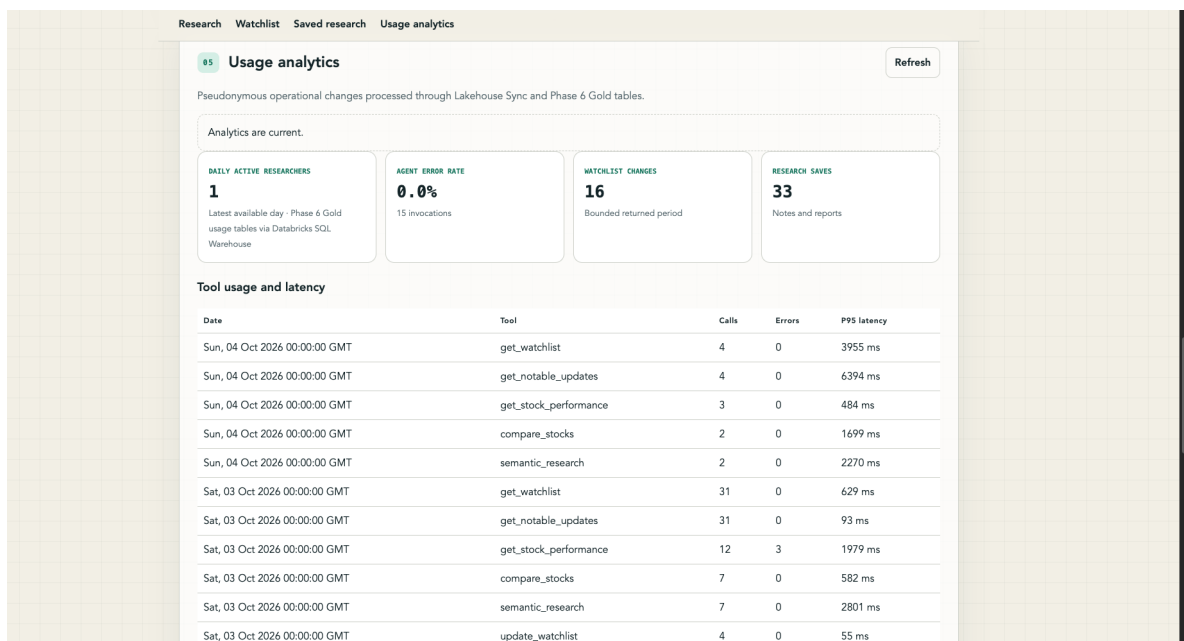


Figure 4 Usage analytics from Databricks Gold tables

The screenshot shows current Gold metrics, including one daily active researcher, a 0.0 percent agent error rate over 15 invocations, 16 watchlist changes, 33 research saves, and per-tool P95 latency.

My Workspace

signal-desk-mcp

Deploys

Search

+ New

Upgrade

?

M

Dashboard

signal-desk-mcp

Deploys

Settings

MONITOR

Events

Logs

Metrics

HANAGE

Compute

Environment

Shell

Previews

Disk

One-Off Jobs

Changelog

Status

Collapse

WEB SERVICE

signal-desk-mcp

Python 3

Free

Upgrade your instance

Blueprint managed

Service ID: snv-daqu5ef0tbc7389kx0

srinivas-malyala / signal-desk-market-research

codex/render-app-deployment

9a2c0ee

Live

https://signal-desk-mcp.onrender.com

Connect

Manual Deploy

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Upgrade now

Search deploys and commits

DEPLOY	TRIGGER	DURATION
fix: self-heal watchlist synchronization 9a2c0ee Deployed 5h ago by you	Manual	1m26s
fix: hydrate watchlist market data cb945da Deployed 8h ago by you	Manual	1m28s
fix: restore attributable research workflows 12a192d Deployed 6h ago by you	Manual	1m25s
fix: preserve performance history when snapshot fails 95711d Deployed 7h ago by you	Manual	1m25s
feat: deploy and evaluate P5 supervisor 451c789 Deployed 3d ago by you	Manual	1m35s
fix: serialize all action timestamps 357d4dc Deployed 3d ago	Health check path updated	49.6s
fix: serialize all action timestamps 357d4dc Deployed 3d ago by you	Manual	1m27s

Figure 5 Live Render MCP deployment

Render links the service to the GitHub repository and deployed commit. The visible Free plan warning is an operational constraint, not a functional failure; the runbook requires pre-warming before a demonstration.

Verification and Release Evidence

The submission combines current local verification with accepted workspace and deployed evidence. Local verification was rerun immediately before producing this package.

Check	Result	Meaning
Automated tests	285 passed	Unit, contract, route, safety, pipeline-source, deployment, and acceptance behavior
Static analysis	Ruff passed	Repository-wide Python lint checks
Supervisor evaluation	10 of 10 passed	Deployed action-taking agent and tool selection
Retrieval evaluation	51 cases accepted	High recall, ranking quality, provenance, and metadata filtering
Credential scan	Accepted release checkpoint	No secrets in tracked source or sanitized evidence
Live app inspection	Verified	Authenticated research, state, and analytics views loaded

Five Minute Demonstration Path

1. Open the MCP URL first, then the frontend, and confirm the authenticated researcher.
2. Run an AAPL filing and news question; inspect source type, date, score, and source URL.
3. Run ticker performance and a two-company comparison on a shared window.
4. Demonstrate a confirmed, disposable watchlist write and explain idempotent replay protection.
5. Show research memory and the usage analytics metrics derived from Lakebase changes.
6. Finish with the architecture diagram, the Volume and Variety counts, the Live Render deployment, and the 285-test result.

Operational Constraints

- Render Free instances can sleep after inactivity. Pre-warm MCP first and frontend second before a live demo.
- Analytics may lag current UI actions. The interface labels stale data and shows the latest processed result.
- Massive plan entitlements can limit snapshots or fundamentals. The application labels unavailable data and falls back only through documented paths.
- The application supports research and evidence inspection, not trade execution or personalized investment advice.

Submission Checklist

- Upload Signal_Desk_Capstone_Submission.zip when submitting the complete evidence package.
- Upload Signal_Desk_Capstone_Evidence.pdf instead when the grader prefers one directly viewable file.
- Confirm the live application URL opens and the approved Google account can authenticate.
- Pre-warm the MCP and frontend Render services before recording or presenting the demo.
- Keep the five-minute demo script open and remove any disposable write created during the presentation.

Evidence File Index

The repository includes detailed phase status, architecture decisions, runbooks, tool references, and acceptance logic. The files below are the shortest path for a grader who wants to inspect the implementation behind a claim.

Evidence need	Files
Concise requirement map	docs/release/REQUIREMENTS_TRACEABILITY.md
Complete implementation history	docs/IMPLEMENTATION_STATUS.md
Deployment procedure	docs/RENDER_DEPLOYMENT_RUNBOOK.md and render.yaml
Architecture	README.md and proposal/signal-desk-capstone-architecture.svg
Data contracts	shared/contracts/models.py and docs/release/DATA_DICTIONARY.md
MCP tool behavior	mcp_server/stock_research_mcp_server.py and docs/release/TOOL_API_REFERENCE.md
Agent evaluation	agent/evaluations.json and tools/phase7_agent_eval.py
Spark definitions	pipelines/ and resources/*.pipeline.yml
Lakebase schema	mcp_server/migrations/ and mcp_server/lakebase.py
Analytics acceptance	tools/phase6_cdf_acceptance.py and pipelines/gold_*
Release checklist	docs/release/DEMO_CHECKLIST.md
Submission navigation	submission/EVIDENCE_MANIFEST.md and submission/DEMO_SCRIPT.md

Submission Links

[Signal Desk frontend](#)

[Signal Desk MCP service](#)

[Source repository](#)

Prepared for the DataExpert Databricks AI Capstone rubric. All point claims are tied to observable behavior or a named repository artifact.